

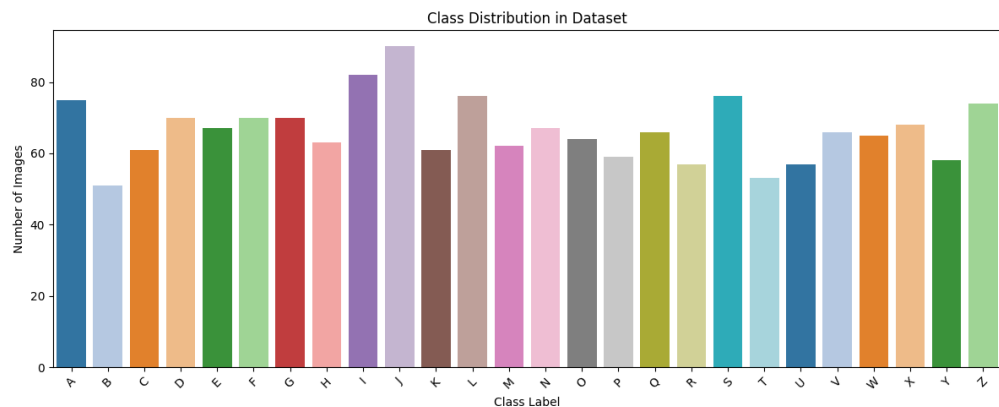
Sign Language

در این پروژه هدف طراحی سیستمی است که بتواند حروف الفبای انگلیسی را از روی تصاویر Sign Language تشخیص دهد، دیتاست شامل تصاویری از ژست‌های مختلف دست است. ابتدا ساختار کلی دیتاست و نحوه تخصیص داده‌ها به بخش‌های مختلف (Split) بررسی می‌شود.

جدول ۱- نتایج وضعیت مجموعه‌های داده

همپوشانی	درصد از کل	تعداد نمونه	Split مجموعه داده
•	۸۷.۵٪	۱۵۱۲	train
•	۸.۳٪	۱۴۴	validation
•	۴.۲٪	۷۲	test
–	۱۰۰٪	۱۷۲۸	total

در جدول ۱، تقسیم‌بندی داده‌ها به صورت (۸۷.۵٪) $train=1512$ ، (۸.۳٪) $validation=144$ و (۴.۲٪) $test=72$ انجام شده است. بررسی هم‌پوشانی نشان می‌دهد هیچ تداخلی بین مجموعه‌ها وجود ندارد؛ بنابراین Leakage بین مجموعه‌های ترین و ولیدیشن و تست اتفاق رخ نمی‌دهد. با وجود نبود data Leakage، تقسیم‌بندی داده‌ها رویکردی train heavy دارد؛ زیرا به دلیل محدود بودن حجم دیتاست، بخش عمده‌ای از داده‌ها به آموزش و مجموعه‌ی test نسبتاً کوچک در نظر گرفته شده است. در مجموعه test، ۷۲ نمونه برای ۲۶ کلاس ممکن است باعث نوسان در معیارهای نهایی عملکرد شود. می‌توان یا حجم مجموعه‌ی test را کمی افزایش داد، یا از روش‌هایی مانند cross validation استفاده کرد ولی ساختار تقسیم‌بندی در این تمرین از قبل مشخص شده است، بنابراین در ادامه‌ی کار تحلیل و ارزیابی را بر اساس همین ساختار موجود پیش برده می‌شود.



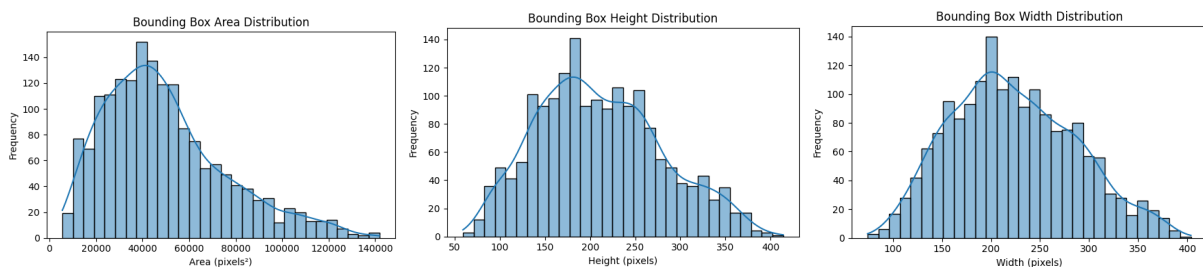
شکل ۱- نمودار میله‌ای توزیع فراوانی کلاس‌ها در مجموعه دیتا

در شکل ۱، نمودار میله‌ای توزیع فراوانی کلاس‌ها که تعداد تصاویر موجود برای هر یک از ۲۶ کلاس نشان می‌دهد که در حالت Balanced Dataset است. اختلاف بین بیشترین تکرار کلاس J با ۹۰ نمونه و کمترین تکرار کلاس B با ۵۱ نمونه باعث ایجاد چولگی شدید در آموزش نمی‌شود و توزیع یکنواخت نیز نشان‌دهنده‌ی مدل حین یادگیری دچار بایاس برای حروف خاص نمی‌شود.

جدول ۲- نتایج توصیفی ابعاد و مساحت Bounding Box

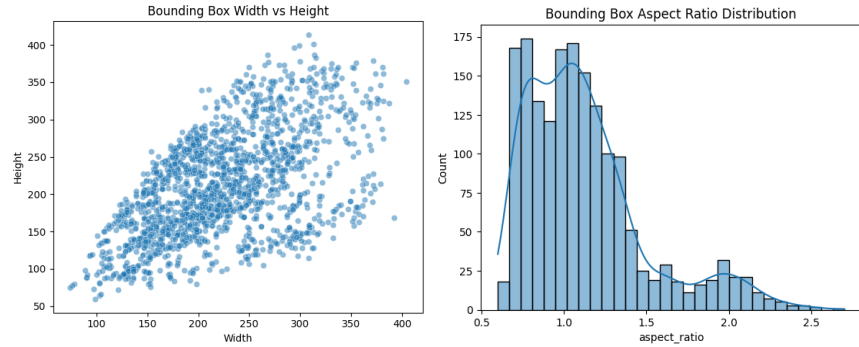
شاخص آماری	عرض	ارتفاع	مساحت
تعداد کل	۱۷۲۸	۱۷۲۸	۱۷۲۸
میانگین	۲۲۲.۶	۲۱۰.۷	۴۹۳۴۹.۸
انحراف معیار	۶۳.۵	۶۹.۲	۲۶۲۹۴.۹
کمینه	۷۴.۰	۵۹.۰	۵۵۵۰.۰
میانه	۲۱۷.۰	۲۰۳.۰	۴۴,۴۶۴.۵
بیشینه	۴۰۴.۰	۴۱۴.۰	۱۴۱۸۰۴.۰

براساس جدول ۲، مقادیر میانگین به میانه در هر دو پارامتر عرض (۲۲۲ و ۲۱۷) و ارتفاع (۲۱۰ و ۲۰۳) نشان‌دهنده یک توزیع متقارن و یکنواخت در ابعاد اشیاء است که نشان می‌دهد تصاویر در شرایط فاصله‌ای مشابهی از دوربین ثبت شده‌اند. همچنین اختلاف میان حداقل مساحت ۵۵۵۰ و حداکثر مساحت ۱۴۱۸۰۴ نشان‌دهنده تنوع مقیاس در دیتاست است. اشیایی با مساحت‌های بسیار بالا احتمالاً مربوط به تصاویر بسیار نزدیک هستند که بخش بزرگی از کادر را پوشانده‌اند. با توجه به انحراف معیار حدود ۶۰ تا ۶۹.۵ پیکسلی در ابعاد، مدل باید توانایی تشخیص اشیاء در سایزهای مختلف را داشته باشد.



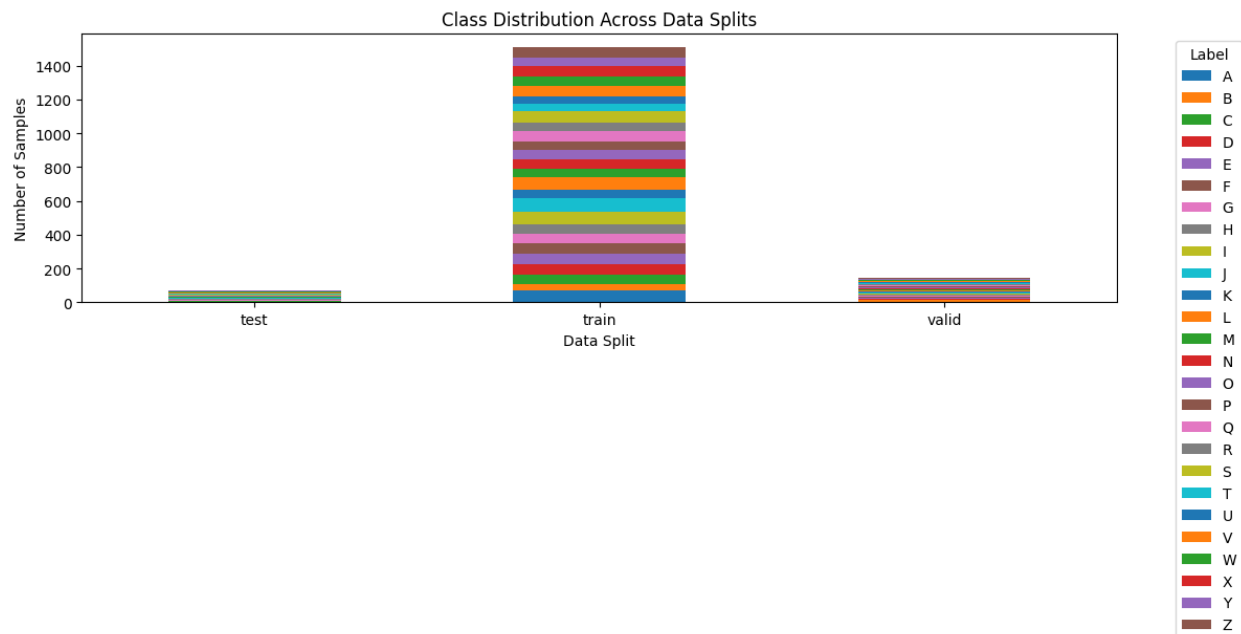
شکل ۲- نمودارهای تحلیل توزیع ابعاد و مساحت Bounding Box

شکل ۲ نشان می‌دهد که عرض و ارتفاع تصاویر بیشتر در بازه ۱۵۰ تا ۳۰۰ پیکسل متمرکز بوده است و توزیعی متقارن با نسبت ابعاد نزدیک به مربع دارند؛ همچنین مساحت باکس‌ها بین ۲۰ تا ۶۰ هزار پیکسل مربع متغیر است که بیانگر تفاوت فواصل کاربر از دوربین در دیتاست می‌باشد. بنابراین به‌کارگیری استراتژی padding هنگام تغییر سایز به 128×128 را بلامانع است زیرا این روش علاوه بر حفظ نسبت ابعاد و جلوگیری از دفرمه شدن سوژه، جزئیات ظریف انگشتان را برای معماری‌های CNN بهینه نگه داشته و پایداری مدل در برابر تغییرات مقیاس را تضمین می‌کند.



شکل ۳- نمودار پراکندگی عرض و ارتفاع باکس‌ها و نمودار توزیع نسبت ابعاد

براساس شکل ۳، نمودار پراکندگی رابطه بین عرض و ارتفاع باکس‌ها، همبستگی مثبت و خطی قوی‌ای را نشان می‌دهد که گویای تناسب فیزیکی دست در زوایای مختلف است؛ در کنار آن، نمودار توزیع نسبت ابعاد نشان می‌دهد که اکثر داده‌ها در بازه نزدیک به ۱ قرار دارند، اما پراکندگی تا مقادیر ۲ و کمی بالاتر نیز دیده می‌شود. این تنوع در نسبت ابعاد، مجدد استفاده از تکنیک **Padding** هنگام تغییر سایز تصاویر به 128×128 را نمایان می‌کند؛ زیرا که بدون **Padding**، تغییر سایز مستقیم منجر به کشیدگی توزیع در تصاویر با نسبت ابعاد غیرمربعی می‌شود که باعث کاهش دقت مدل در استخراج ویژگی‌های کلیدی دست‌ها می‌شود.



شکل ۴- نمودار توزیع کلاس‌ها در بخش‌های مختلف دیتاست

در شکل ۴، نمودار توزیع کلاس‌ها در بخش‌های مختلف دیتاست (train, validation, test) نشان می‌دهد که کاملاً تقسیم‌بندی متوازن است. با مشاهده ارتفاع یکسان بلوک‌های رنگی مربوط به هر کلاس در هر سه بخش، می‌توان نتیجه گرفت که استراتژی نمونه‌برداری به‌درستی انتخاب شده است زیرا نسبت فراوانی هر حرف الفبا در مجموعه‌های آموزش، اعتبارسنجی و تست یکسان باقی مانده است.

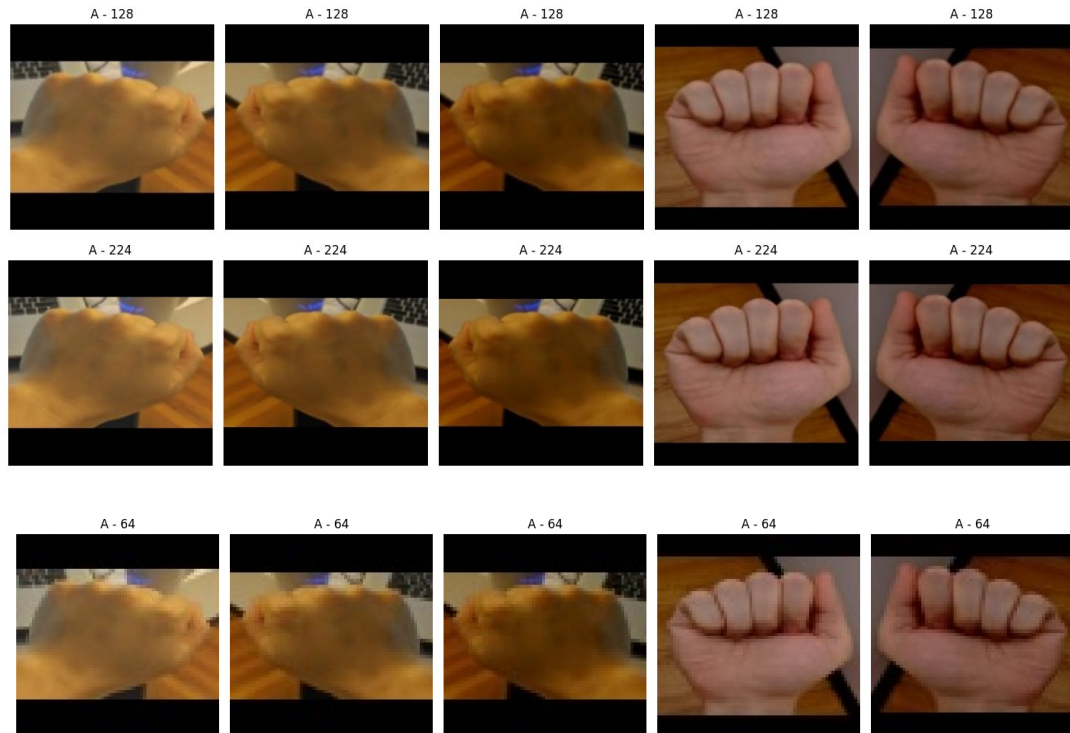
جدول ۳- خلاصه آماری توزیع کلاس‌ها در بخش‌های دیتاست

نرخ عدم توازن	Sd	Mean	Max	Min	Split
۲.۰	۱۰.۰۸	۵۸.۱۵	۷۸	۳۹	Train
۹.۰	۲.۱۸	۵.۵۴	۹	۱	Valid
N/A	۱.۴۵	۲.۷۶	۵	۰	Test

در جدول ۳، پایداری مناسب Train با نرخ عدم توازن ۲ است که نشان‌دهنده دسترسی کافی مدل به تمامی کلاس‌هاست. در عوض انحراف معیار بالاتر و نرخ عدم توازن متغیر در مجموعه‌های validation و test، نشان می‌دهد که حجم نمونه در مجموعه‌های ارزیابی کم است بنابراین، در تحلیل عملکرد نهایی می‌بایست در نظر داشت که برخی خطاهای مدل ممکن است از تعداد کم در نمونه‌های ارزیابی در برخی کلاس‌های خاص باشد.

بخش ب) Data Preprocessing

برای آماده‌سازی مجموعه داده‌ی تصاویر ابتدا از تابع `crop_hand_images` استفاده می‌شود. فرایند استخراج با مرتب‌سازی دیتافریم حاوی اطلاعات تصاویر و مختصات Bounding Box آغاز می‌شود. سپس پوشه‌ای به نام `cropped` برای ذخیره‌سازی تصاویر پس از برش ایجاد می‌گردد. برای هر تصویر در مجموعه داده مختصات Bounding Box استخراج شده و با استفاده از این مختصات، ناحیه‌ی مشخص شده از تصویر اصلی برش داده می‌شود. در گام بعدی باید برای ورود به مدل به اندازه‌های استاندارد و یکسانی تبدیل شوند بنابراین از تابع `resize_with_padding` این فرایند را استفاده می‌شود. به این ترتیب که ابتدا تصویر را با حفظ نسبت ابعاد اصلی خود، به بزرگترین اندازه‌ای که در قاب هدف جای گیرد، تغییر اندازه می‌دهد. سپس، فضاهای خالی باقی‌مانده در قاب هدف را با رنگ سیاه پر می‌کند. این روش از کشیدگی و تصویر جلوگیری کرده و ابعاد ثابتی را برای تمام تصاویر ورودی مدل فراهم می‌آورد. تابع `resize_images_in_folder` این فرآیند را به صورت دسته‌ای برای کل مجموعه داده‌ها اجرا می‌کند. این تابع برای هر مجموعه داده، پوشه‌های جداگانه‌ای برای اندازه‌های مختلف ایجاد کرده و تصاویر برش‌خورده را پس از تغییر اندازه و اعمال `padding` در این پوشه‌ها سازماندهی می‌کند که تمام تصاویر ورودی به مدل، دارای ابعاد یکسان و فرمت مناسبی باشند. (به علت افلاین کد زدن و چک کردن با توجه به حجم دیتاست تصاویر سیو شده است، اما می‌دانم حجم دیتاست بیشتر باشد این کار توصیه نمی‌شود).



شکل ۵ - نمونه ای از تصاویر پس از اعمال روند cropping و resize به همراه padding

در شکل ۵، تصاویر دست پس از اعمال این مرحله در اندازه‌های مختلف 64×64 ، 128×128 و 224×224 ذخیره شده‌اند و ۵ نمونه به صورت تصادفی نمایش داده شده است، اما در عین حال نسبت ابعاد اصلی تصویر تا حد امکان حفظ شده است. شکل کلی دست در همه‌ی اندازه‌ها تقریباً ثابت مانده و فقط وضوح و جزئیات تصویر با بزرگ‌تر شدن اندازه بیشتر شده است. وجود نوارهای مشکی برای جلوگیری از کشیدگی تصویر، فضای خالی با Padding پر شده است، بنابراین روند به خوبی طی شده است. ابعاد 128×128 به عنوان نقطه بهینه انتخاب شد زیرا این اندازه توازن بین حفظ جزئیات ساختاری دست و مدیریت زمان برای محاسبات می باشد. اما وضوح 64×64 منجر به از دست رفتن ویژگی‌های ظریف بافت و فرم انگشتان می‌شود، انتخاب ابعاد 224×224 نیز علاوه بر افزایش بی‌رویه بار پردازشی، ریسک overfitting را در صورت محدود بودن داده‌ها افزایش می‌دهد. اما برخی مدل‌ها با ابعاد 224×224 و 64×64 بهتر عمل می‌کنند اما برای نقطه بهینه در این پروژه 128×128 انتخاب شده است. (البته واقعاً متوجه ام باید تست می‌کردم اما به علت افلاکین کد زدن وزن های ۱۲۸ مناسب مدل فقط توانستم دانلود کنم و دستی وارد کدم کنم و با این دلایل پاراگراف به همین بسنده کردم).

سپس با توجه به روند پروژه، عملیات نرمال‌سازی انجام می‌شود تا مقادیر پیکسل‌ها که در بازه‌ی [۰،۲۵۵] هستند، به بازه‌ی [۰،۱] تبدیل شوند. این کار با تبدیل نوع داده به float^{32} و تقسیم مقادیر بر ۲۵۵ انجام شده و باعث می‌شود آموزش مدل پایدارتر و همگرا شدن سریع‌تر شود. برای افزایش سرعت و کارایی در مراحل بعدی تصویر نرمال‌سازی شده به جای ذخیره به صورت فایل تصویری، در قالب آرایه numpy و با فرمت npy ذخیره شد.

داده‌ها با تابع `load_dataset` بارگذاری می‌شوند؛ به این صورت که فایل‌های `.npy` هر کلاس خوانده شده، در صورت نیاز به قالب مناسب تبدیل می‌گردند و سپس با تبدیل فضای رنگی از BGR به RGB، سازگاری رنگ‌ها حفظ می‌شود. پس از بارگذاری مجموعه‌های `train, validation, test` داده‌های آموزش `Shuffle` می‌شوند تا ترتیب نمونه‌ها روی یادگیری اثر نگذارد. در نهایت با استفاده از `LabelEncoder`، برچسب‌های متنی به برچسب‌های عددی تبدیل می‌شوند و برای اینکه نگاشت کلاس‌ها در هر سه مجموعه یکسان بماند، `Encoder` روی مجموع برچسب‌های `Train, Valid, Test` فیت شده و تعداد کلاس‌ها نیز از روی کلاس‌های نهایی محاسبه می‌شود. در این پروژه دو مدل مورد ارزیابی قرار گرفته شده است. در اولین بخش معماری `EfficientNetB0` مورد استفاده قرار گرفته است، تصاویر ورودی که قبلاً در بازه `[۰, ۱]` نرمال شده بودند، در مقیاس `[۰, ۲۵۵]` قرار گرفتند تا توزیع داده‌ها با انتظارات مدل پیش‌آموزش‌دیده سازگار باشد. به منظور افزایش تنوع داده و کاهش بیش‌برازش، یک بلوک `Data Augmentation` شامل چرخش، زوم، جابه‌جایی، تغییر کنتراست و روشنایی به ابتدای شبکه اضافه شد تا مدل در برابر تغییرات رايج تصویری مقاوم‌تر شود. در گام اول کل بدنه‌ی `EfficientNet` فریز شد (`trainable=False`) و فقط یک `head` طبقه‌بندی شامل `GlobalAveragePooling`، `Dropout` و لایه‌ی `Dense` با فعال‌سازی `Softmax` (به همراه `۲L regularization`) آموزش داده شد. مدل با `Adam` و نرخ یادگیری `۱۰-۳` و برای کنترل روند یادگیری از `EarlyStopping` و `ReduceLROnPlateau` بر اساس `val_loss` و همچنین `ModelCheckpoint` برای ذخیره بهترین مدل بر اساس `val_accuracy` استفاده شد. در گام دوم نیز برای بهبود عملکرد فرآیند `Finetuning` انجام شد کل بدنه `EfficientNet` قابل آموزش شد، اما لایه‌های ابتدایی تا حدود ۳۰ لایه آخر فریز باقی ماندند تا ویژگی‌های عمومی حفظ شوند و فقط بخش‌های عمیق‌تر روی داده جدید تنظیم شوند. همچنین تمام لایه‌های `BatchNormalization` غیرقابل آموزش نگه داشته شدند، چون در `Fine-tuning` خصوصاً با داده کم و `Batch` کوچک تغییر آمار میانگین و واریانس می‌تواند باعث ناپایداری و افت تعمیم‌پذیری شود. در این مرحله مدل دوباره با نرخ یادگیری بسیار کوچک‌تر `۱۰-۵` کامپایل شد تا تغییرات وزن‌ها ملایم و کنترل‌شده باشد و همان `Callbacks` برای جلوگیری از `overfitting` و ذخیره بهترین وزن‌ها استفاده گردید.

جدول ۴- نتایج در گام دوم آموزش `Fine-tuning` مدل `EfficientNetB0`

Best Epoch (Saved by Val Accuracy): ۳۵

--- Metrics at Best Saved Model Epoch ---

Train Accuracy : ۰.۷۸۹۰

Valid Accuracy : ۰.۷۱۵۳

Train Loss : ۰.۶۹۱۳

Valid Loss : ۰.۹۴۳۲

--- Absolute Peaks in History ---

Absolute Max Train Accuracy : ۰.۸۰۴۹ (at Epoch ۴۱)

Absolute Max Valid Accuracy : ۰.۷۱۵۳ (at Epoch ۳۵)

Absolute Min Train Loss : ۰.۶۵۴۳ (at Epoch ۴۵)

Absolute Min Valid Loss : ۰.۹۳۸۹ (at Epoch ۴۴)

--- Last Epoch of Training (Before stopping) ---

Final Train Accuracy : ۰.۸۰۲۹

Final Valid Accuracy : ۰.۷۰۱۴

Final Train Loss : ۰.۶۶۲۱

Final Valid Loss : ۰.۹۳۹۴

در جدول ۴، نتایج در گام دوم آموزش Fine-tuning مدل EfficientNetB۰ نشان می‌دهد که بهترین مدل بر اساس بیشترین val_accuracy در epoch=۳۵ حاصل شده است؛ به‌طوری‌که که در این نقطه $\text{train accuracy} = ۰.۷۸۹۰$ و $\text{validation accuracy} = ۰.۷۱۵۳$ و $\text{train Loss} = ۰.۶۹۱۳$ و $\text{validation Loss} = ۰.۹۴۳۲$ می‌باشد. همچنین کمترین مقدار val_loss در epoch=۴۴ مشاهده شد. اما با توجه به معیار انتخاب بهترین مدل همان epoch = ۳۵ به عنوان بهترین مدل نظر گرفته شد.

جدول ۵- عملکرد کلی مدل روی مجموعه‌ی Test در مدل EfficientNetB۰

Metric	Value
Test Loss	۱,۲۰۵۹
Test Accuracy	۰,۶۲۵۰
Macro Avg Precision	۰,۶۶
Macro Avg Recall	۰,۶۳
Macro Avg F1-score	۰,۵۸
Weighted Avg Precision	۰,۷۰
Weighted Avg Recall	۰,۶۲
Weighted Avg F1-score	۰,۶۲
Test Samples (Support)	۷۲

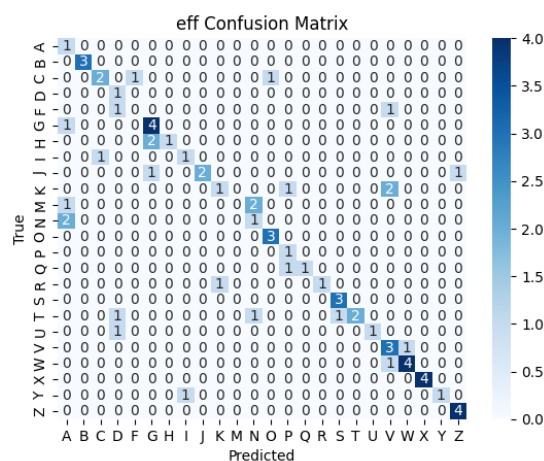
در جدول ۵، مدل EfficientNetB۰ روی مجموعه‌ی Test به $\text{Test Accuracy} = ۰,۶۲۵۰$ و $\text{Test Loss} = ۱,۲۰۵۹$ رسیده است. همچنین مقادیر میانگین‌ها $\text{Weighted F1} = ۰,۶۲$ تقریباً هم‌سطح با Accuracy است و نشان می‌دهد عملکرد کلی مدل در سطح قابل قبول قرار دارد. در مقابل، $\text{Macro F1} = ۰,۵۸$ کمتر از Weighted F1 است؛ این اختلاف معمولاً بیانگر آن است که عملکرد مدل در همه‌ی کلاس‌ها یکنواخت نیست و در برخی کلاس‌ها مخصوصاً کلاس‌هایی که نمونه‌ی کمتری دارند، ضعیف‌تر بوده است. همچنین با توجه به اینکه تعداد مجموعه‌ی Test، ۷۲ نمونه است؛ این نتایج کلی تا حدی قابل قبول هستند.

جدول ۶- گزارش Classification برای هر کلاس در مدل EfficientNetB۰

Class	Precision	Recall	F1-score	Support
A	۰.۲۰	۱.۰۰	۰.۳۳	۱
B	۱.۰۰	۱.۰۰	۱.۰۰	۳
C	۰.۶۷	۰.۵۰	۰.۵۷	۴
D	۰.۲۵	۱.۰۰	۰.۴۰	۱
F	۰.۰۰	۰.۰۰	۰.۰۰	۲
G	۰.۵۷	۰.۸۰	۰.۶۷	۵
H	۱.۰۰	۰.۳۳	۰.۵۰	۳
I	۰.۵۰	۰.۵۰	۰.۵۰	۲
J	۱.۰۰	۰.۵۰	۰.۶۷	۴
K	۰.۵۰	۰.۲۵	۰.۳۳	۴
M	۰.۰۰	۰.۰۰	۰.۰۰	۳

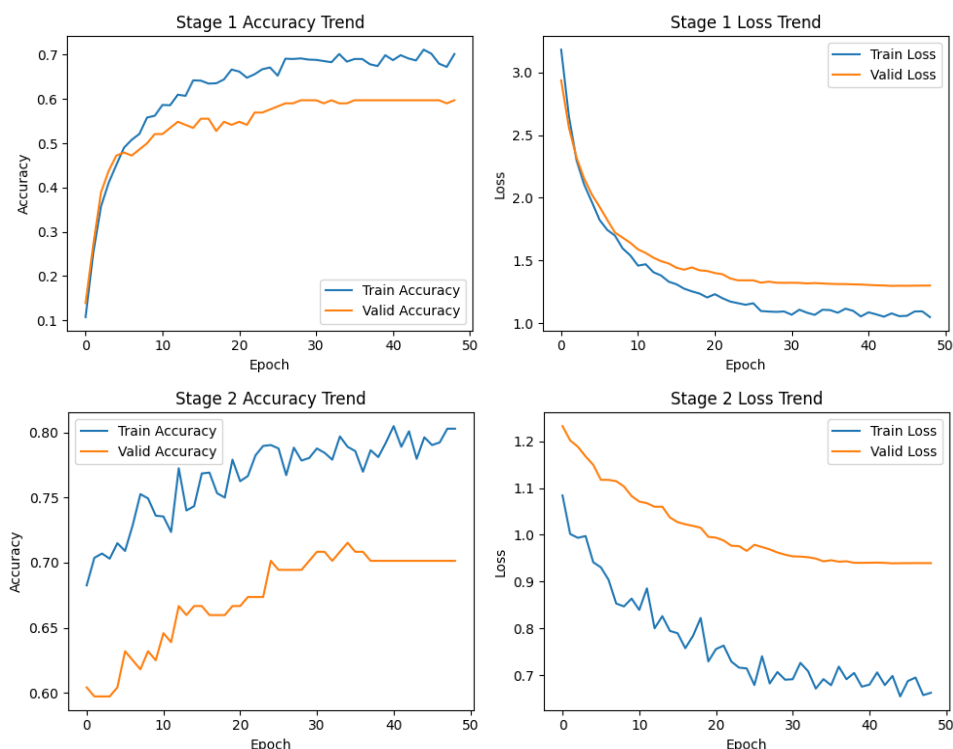
Class	Precision	Recall	F1-score	Support
N	۰.۲۵	۰.۳۳	۰.۲۹	۳
O	۰.۷۵	۱.۰۰	۰.۸۶	۳
P	۰.۳۳	۱.۰۰	۰.۵۰	۱
Q	۱.۰۰	۰.۵۰	۰.۶۷	۲
R	۱.۰۰	۰.۵۰	۰.۶۷	۲
S	۰.۷۵	۱.۰۰	۰.۸۶	۳
T	۱.۰۰	۰.۴۰	۰.۵۷	۵
U	۱.۰۰	۰.۵۰	۰.۶۷	۲
V	۰.۴۳	۰.۷۵	۰.۵۵	۴
W	۰.۸۰	۰.۸۰	۰.۸۰	۵
X	۱.۰۰	۱.۰۰	۱.۰۰	۴
Y	۱.۰۰	۰.۵۰	۰.۶۷	۲
Z	۰.۸۰	۱.۰۰	۰.۸۹	۴

در جدول ۶، مقدار precision، recall و F1-score را برای هر کلاس به همراه تعداد نمونه‌ها نشان می‌دهد. بعضی کلاس‌ها عملکرد خیلی خوبی دارند، همانند کلاس‌های B و X با $F1\text{-score} = 1$ بهترین نتیجه را دارند. همچنین کلاس‌های S, O, Z و W هم $F1\text{-score}$ بالایی دارند و مدل این کلاس‌ها را دقیق‌تر تشخیص داده است. همچنین کلاس‌های F و M دارای $F1\text{-score}$ = ۰ هستند و این یعنی مدل در داده‌های تست نتوانسته این دو کلاس را به درستی شناسایی کند. همچنین باید توجه کرد که بعضی کلاس‌ها تعداد نمونه‌ی خیلی کمی دارند؛ مثلاً کلاس‌های A, D و P فقط یک نمونه دارند. به همین دلیل، مقادیر precision و recall برای این کلاس‌ها ممکن است خیلی ناپایدار باشد و نتوان با اطمینان زیاد درباره‌ی آن‌ها نظر داد. بنابراین برای بررسی دقیق‌تری بایست نمونه‌های بیشتری از این کلاس‌ها داشته باشیم.



شکل ۶- ماتریس Confusion برای مدل EfficientNetB0

شکل ۶ نشان می‌دهد که مدل EfficientNetB۰ در اغلب کلاس‌ها پیش‌بینی‌های درستی داشته و مقادیر روی قطر اصلی ماتریس نسبتاً قابل توجه هستند. کلاس‌های B, O, S, W, X و Z با تعداد پیش‌بینی‌های درست بیشتر عملکرد مناسبی را نشان می‌دهند و بیانگر توانایی مدل در تشخیص این کلاس‌هاست. در برخی کلاس‌ها مانند F, M, N و K خطاهای طبقه‌بندی بیشتری مشاهده می‌شود و بخشی از نمونه‌ها به کلاس‌های نزدیک یا مشابه اشتباه شده‌اند. همچنین در چند کلاس با تعداد نمونه‌ی کم، نوسان پیش‌بینی‌ها بیشتر است که نشان می‌دهد عملکرد مدل روی همه‌ی کلاس‌ها یکنواخت نیست.



شکل ۷- نمودار تغییرات Accuracy و Loss در گام اول و دوم آموزش در مدل EfficientNetB۰

بر اساس شکل ۷، نمودار Loss نشان می‌دهد مدل در طول epochها کم‌کم بهتر یاد گرفته و روند یادگیری‌اش پایدار بوده است. در نمودار Accuracy هم دیده می‌شود که دقت Train و Validation در چند epoch اول با سرعت زیاد بالا می‌رود، اما بعد از آن رشدش کندتر می‌شود و تقریباً به حالت ثابت نزدیک می‌شود. در پایان این مرحله، دقت آموزش حدود ۰.۷۰ و دقت اعتبارسنجی حدود ۰.۶۰ است. در نمودار Loss، هم برای آموزش و هم برای اعتبارسنجی یک کاهش واضح دیده می‌شود؛ یعنی loss از مقدارهای بالاتر به مقدارهای پایین‌تر رسیده و این نشان می‌دهد مدل دارد درست یاد می‌گیرد. همچنین چون فاصله‌ی بین منحنی‌های Train و Validation خیلی زیاد نیست و تقریباً ثابت مانده، می‌توان گفت مدل خیلی overfit نشده و تا حدی توانسته خوب تعمیم بدهد. در ردیف دوم مدل در بخش دوم مرحله آموزش است بطوریکه در نمودار Accuracy، دقت آموزش از حدود ۰.۶۹ شروع می‌شود و کم‌کم به حدود ۰.۸۰ نزدیک می‌شود و هم‌زمان دقت اعتبارسنجی هم از حدود ۰.۶۰ بالا می‌رود و به حدود ۰.۷۱ می‌رسد و از میانه‌های آموزش به بعد در همین محدوده ثابت و با نوسان کم ادامه پیدا می‌کند. این یعنی وقتی چند لایه‌ی آخر را آزاد کردیم، مدل توانسته خودش را بهتر با داده‌های مسئله هماهنگ کند و یادگیری بهتری داشته باشد. در نمودار Loss هم برای هر دو مجموعه کاهش دیده می‌شود؛ به‌طوریکه Train Loss تا حدود ۰.۷۰ پایین می‌آید و Validation Loss هم تا حدود ۰.۹۴ کاهش پیدا می‌کند و بعد تقریباً ثابت می‌شود.

برای اینکه ارزیابی دقیق‌تری داشته باشیم و فقط به نتایج یک مدل بسنده نکنیم، در کنار EfficientNetB۰ یک مدل دیگر هم برای مقایسه آموزش داده شد. با توجه به نتایج حاصل شده برای این مقایسه از MobileNetV۲ استفاده شد که یک مدل سبک و پرکاربرد در دسته‌بندی تصاویر است. ابتدا داده‌ها با تابع پیش‌پردازش اختصاصی MobileNetV۲ نرمال‌سازی شدند. برای افزایش توان تعمیم‌پذیری و کاهش بیش‌برازش، از افزایش داده‌ی ملایم شامل چرخش محدود، زوم کم و جابه‌جایی‌های جزئی استفاده شد تا تنوع نمونه‌های آموزشی بدون تغییر ماهیت کلاس‌ها بیشتر شود.

در گام اول آموزش، بخش پایه‌ی MobileNetV با وزن‌های از پیش‌آموزش‌دیده بارگذاری و فریز شد؛ بنابراین در این فاز فقط لایه‌های افزوده‌شده‌ی انتهایی امکان به‌روزرسانی وزن‌ها را داشتند. سپس برای استخراج بردار ویژگی فشرده، از GlobalAveragePooling استفاده شد و جهت کنترل پیچیدگی مدل، یک Dropout به‌همراه یک لایه‌ی Dense با منظم‌سازی L۲ به‌عنوان head طبقه‌بندی اضافه گردید. مدل با Adam و نرخ یادگیری 10^{-3} آموزش داده شد. برای پایداری روند یادگیری و جلوگیری از ادامه‌ی آموزش در شرایط غیربهبود، از EarlyStopping و از ReduceLrOnPlateau برای کاهش خودکار نرخ یادگیری هنگام توقف بهبود، و از ModelCheckpoint برای ذخیره‌ی بهترین وزن‌ها بر اساس Accuracy validation استفاده شد. (تقریباً همانند بخش قبلی با تفاوت جزئی برازش داده است.)

جدول ۷- نتایج در گام دوم آموزش Fine-tuning مدل MobileNetV

===== mobilenet Final Report =====
Best Epoch (Saved by Val Accuracy): ۳
--- Metrics at Best Saved Model Epoch ---
Train Accuracy : ۰,۸۵۱۲
Valid Accuracy : ۰,۷۷۰۸
Train Loss : ۰,۴۴۷۳
Valid Loss : ۰,۹۶۱۳
--- Absolute Peaks in History ---
Absolute Max Train Accuracy : ۰,۸۷۹۰ (at Epoch ۷)
Absolute Max Valid Accuracy : ۰,۷۷۰۸ (at Epoch ۳)
Absolute Min Train Loss : ۰,۳۸۹۰ (at Epoch ۵)
Absolute Min Valid Loss : ۰,۹۲۴۱ (at Epoch ۲)
--- Last Epoch of Training (Before stopping) ---
Final Train Accuracy : ۰,۸۷۹۰
Final Valid Accuracy : ۰,۷۶۳۹
Final Train Loss : ۰,۳۹۲۸
Final Valid Loss : ۰,۹۴۵۱

در جدول ۷، نتایج مرحله‌ی نهایی آموزش مدل MobileNetV۲ نشان می‌دهد که بهترین مدل بر اساس بیشترین مقدار validation accuracy در اپک ۳ حاصل شده است؛ به‌طوری‌که در این نقطه، train accuracy برابر با ۰,۸۵۱۲، validation accuracy برابر با ۰,۷۷۰۸، train loss برابر با ۰,۴۴۷۳ و validation loss برابر با ۰,۹۶۱۳ بوده است. همچنین کمترین مقدار validation loss در اپک ۲ و برابر با ۰,۹۲۴۱ مشاهده شد. با این حال، با توجه به این‌که معیار انتخاب بهترین مدل، بیشترین مقدار validation accuracy بوده است، اپک ۳ به‌عنوان بهترین مدل در نظر گرفته شد.

جدول ۸ - عملکرد کلی مدل روی مجموعه‌ی Test در مدل MobileNetV۲

Metric	Value
Test Loss	۱.۳۲۸۸
Test Accuracy	۰.۶۹۴۴
Macro Avg Precision	۰.۶۶
Macro Avg Recall	۰.۶۹
Macro Avg F۱-score	۰.۶۴
Weighted Avg Precision	۰.۷۱
Weighted Avg Recall	۰.۶۹
Weighted Avg F۱-score	۰.۶۸
Test Samples (Support)	۷۲

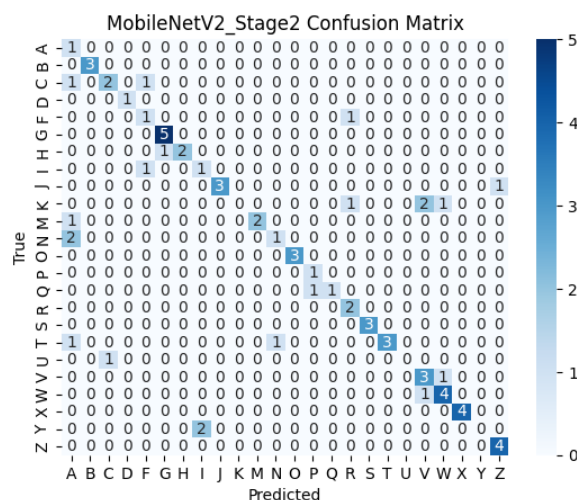
در جدول ۸ نشان می‌دهد که مدل MobileNetV۲ در مرحله‌ی دوم عملکرد قابل قبولی از خودش نشان داده و توانسته روی داده‌های تست به دقت ۶۹.۴۴ درصد و Loss برابر با ۱.۳۲۸۸ برسد که نسبت به مدل قبلی یعنی EfficientNetB۰ یک بهبود محسوس و امیدوارکننده محسوب می‌شود. همچنین نزدیک بودن مقدار $Weighted F1 = 0.68$ به $Macro F1 = 0.62$ نشان می‌دهد که مدل فقط روی برخی کلاس‌های پر تعداد خوب عمل نکرده، بلکه در پیش‌بینی کلاس‌های مختلف نیز تا حد زیادی متعادل تر و پایدارتر بوده است. با توجه به اینکه مجموعه‌ی Test تنها شامل ۷۲ نمونه بوده، همین میزان بهبود می‌تواند کاملاً قابل قبول باشد.

جدول ۹ - گزارش Classification برای هر کلاس در مدل MobileNetV۲

Class	Precision	Recall	F۱-score	Support
A	۰.۱۷	۱.۰۰	۰.۲۹	۱
B	۱.۰۰	۱.۰۰	۱.۰۰	۳
C	۰.۶۷	۰.۵۰	۰.۵۷	۴
D	۱.۰۰	۱.۰۰	۱.۰۰	۱
F	۰.۳۳	۰.۵۰	۰.۴۰	۲
G	۰.۸۳	۱.۰۰	۰.۹۱	۵
H	۱.۰۰	۰.۶۷	۰.۸۰	۳
I	۰.۳۳	۰.۵۰	۰.۴۰	۲
J	۱.۰۰	۰.۷۵	۰.۸۶	۴
K	۰.۰۰	۰.۰۰	۰.۰۰	۴
M	۱.۰۰	۰.۶۷	۰.۸۰	۳
N	۰.۵۰	۰.۳۳	۰.۴۰	۳

O	۱.۰۰	۱.۰۰	۱.۰۰	۳
P	۰.۵۰	۱.۰۰	۰.۶۷	۱
Q	۱.۰۰	۰.۵۰	۰.۶۷	۲
R	۰.۵۰	۱.۰۰	۰.۶۷	۲
S	۱.۰۰	۱.۰۰	۱.۰۰	۳
T	۱.۰۰	۰.۶۰	۰.۷۵	۵
U	۰.۰۰	۰.۰۰	۰.۰۰	۲
V	۰.۵۰	۰.۷۵	۰.۶۰	۴
W	۰.۶۷	۰.۸۰	۰.۷۳	۵
X	۱.۰۰	۱.۰۰	۱.۰۰	۴
Y	۰.۰۰	۰.۰۰	۰.۰۰	۲
Z	۰.۸۰	۱.۰۰	۰.۸۹	۴

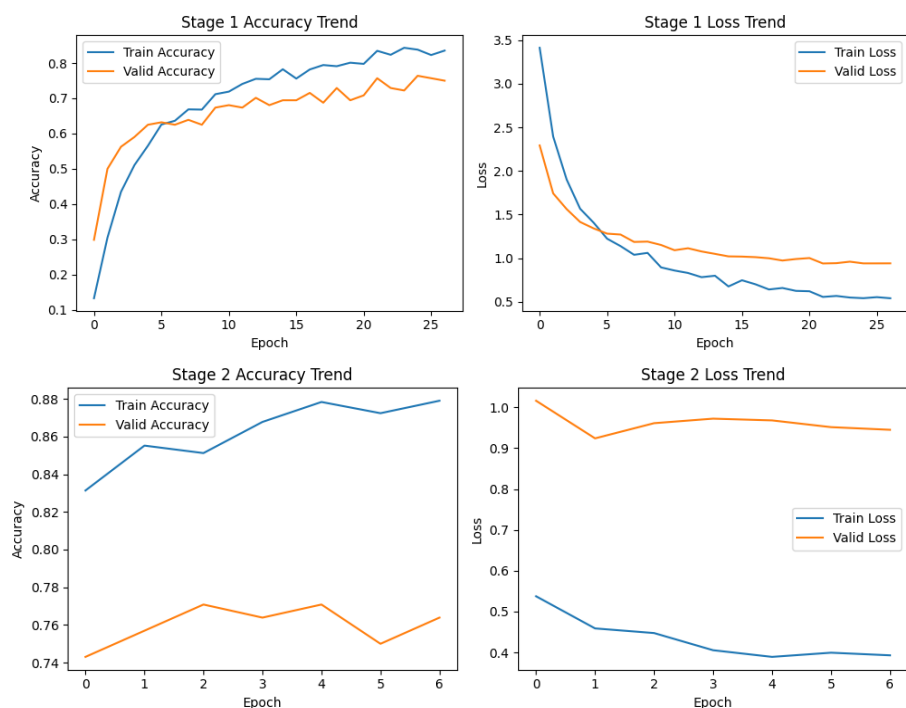
در جدول ۹ نشان می‌دهد مدل MobileNetV۲ در اغلب کلاس‌ها پیش‌بینی‌های درستی داشته است همانند کلاس‌های D, B, O, S و X با $F1\text{-score} = ۱$ عملکردی بی‌نقص داشته‌اند و کلاس‌های J, H, G, M و Z نیز با افوان بالای ۸۰ درصد بسیار دقیق شناسایی شده‌اند. در مقابل، مدل در تشخیص کلاس‌های U, K و Y کاملاً ناموفق بوده و به $F1\text{-score} = ۰$ رسیده است که این ضعف معمولاً به خاطر کم بودن نمونه‌های این کلاس‌ها در داده‌های آموزشی اتفاق می‌افتد. همچنین برای کلاس‌هایی مثل A که فقط یک نمونه دارند، تعادل معیارها به هم خورده است.



شکل ۸- ماتریس Confusion برای مدل MobileNetV۲

شکل ۸ نشان می‌دهد که این مدل نسبت به مدل قبلی بهتر عمل کرده است؛ زیرا تعداد پیش‌بینی‌های درست در اکثر کلاس‌ها همانند G, W, X و Z افزایش یافته است. پررنگ بودن خانه‌های روی قطر اصلی نشان‌دهنده این است که مدل در تمایز قائل شدن بین حروف مختلف، موفق‌تر عمل کرده است. اما در ستون کلاس A، مدل چند نمونه از کلاس‌های دیگر مثل M و N را به اشتباه

A تشخیص داده که باعث شده دقت این کلاس پایین بیاید. همچنین در کلاس‌هایی مثل K و Y، هیچ عددی روی قطر اصلی دیده نمی‌شود که مدل هنوز در شناسایی این کلاس‌های خاص مشکل دارد.



شکل ۹- نمودار تغییرات Accuracy و Loss در گام اول و دوم آموزش در مدل MobileNetV2

بر اساس شکل ۹، در گام اول آموزش در این مدل (ردیف اول) نمودارها نشان می‌دهند که مدل شروع بسیار خوبی داشته است. در نمودار accuracy، دقت دو مجموعه با شیب تندی بالا رفته و بعد از حدود ۲۵ اپوک، train accuracy به بالای ۰.۸۰ و validation accuracy به حدود ۰.۷۵ رسیده است. در نمودار Loss هم یک سقوط سریع و منظم دیده می‌شود که نشان می‌دهد مدل خیلی سریع با ویژگی‌های کلی تصاویر هماهنگ شده است. نزدیکی دو منحنی به هم در این مرحله نشان‌دهنده این است که مدل دچار overfitting نشده و یادگیری پایداری داشته است. در گام دوم آموزش مدل (ردیف دوم) نمودار Accuracy نشان می‌دهد دقت آموزش تا نزدیک ۰.۸۸ هم پیش رفته است، اما دقت validation با نوساناتی در محدوده ۰.۷۷ باقی مانده است. در نمودار Loss مشاهده می‌شود train accuracy که روند کاهشی دارد و به زیر ۰.۴ رسیده، اما Valid Loss کمی بالاتر رفته و حدوداً ۰.۹۵ ثابت شده است.

نتیجه گیری:

در مجموع، با وجود کوچک بودن مجموعه‌ی آزمون، روند آماده‌سازی داده‌ها (برش ناحیه دست، تغییر اندازه با padding و نرمال‌سازی) و آموزش دو مرحله‌ای (آموزش اولیه و سپس fine-tuning) درست بود. EfficientNetB بعد از fine-tuning به عملکرد قابل قبولی رسید، اما نتایج نشان داد که عملکردش بین کلاس‌ها یکنواخت نیست و در بعضی حروف (مثل F و M) تشخیص ناموفق داشت. برای ارزیابی دقیق‌تر MobileNetV هم آموزش داده شد و در مرحله‌ی fine-tuning بهبود قابل توجهی ایجاد کرد؛ به‌طوری که هم تعمیم بهتری داشت و هم پیش‌بینی‌ها در اکثر کلاس‌ها پایدارتر بود. بررسی گزارش کلاس‌به‌کلاس

و ماتریس اغتشاش نیز در مدل MobileNetV در بسیاری از حروف عملکرد قوی‌تری دارد، هرچند چند کلاس مانند K، U و Y همچنان درست نبودند باقی مانده‌اند که احتمالاً به شباهت ژست‌ها، دشواری بصری یا محدودیت داده مربوط است. در نتیجه، MobileNetV به عنوان گزینه‌ی مناسب‌تر برای این مسئله انتخاب می‌شود.